

高精度数*高精度数

有两个不超过240位的正整数用求输出它们相乘的结果，同样也可以使用模拟竖式的方法来实现。

$$\begin{array}{r} 245 \\ 125 \\ \hline 1225 \\ 4900 \\ 2450 \\ \hline 30625 \end{array}$$

图1

如图1所示。假如说245和125是高精度数，那么可以把它们分别倒序存放在数组a，数组b中。这个过程和大数加法一致先用字符数组存放，然后转为整数再倒序存放a,b数组中。

```
1 // 倒序存放
2 void reverse(char* src, int* arr)
3 {
4     for (int i = 0; i < strlen(src); i++)
5     {
6         arr[i] = src[strlen(src) - 1 - i] - '0';
7     }
8 }
9
10
11
12
13 int main()
14 {
15     char s1[200] = { 0 };
16     char s2[200] = { 0 };
17
18     int a[210] = { 0 };
19     int b[210] = {0};
20     scanf("%s %s", s1, s2);
21
```

```

22     reverse(s1, a);
23
24     return 0;
25 }
26

```

紧接着就是用245的每一位和125的5（最低位）相乘，然后把结果保存在数组 `c` 中。经过运算后最后倒着输出数组c的每个元素即可。

第一次算出的结果错位存放数组c中，第二次乘的结果加上c数组中第一次乘的结果；第三次乘的结果加上数组c中第二次的结果。两层循环嵌套，外层循环控制数组的位数内循环控制数组a的位数，因为每次都是拿a的每一位和b的其中一位相乘（从左往右）

那么数组c应该开多大呢？两个不超过240位的数字相乘结果会是多少位呢？我直接问ai了，不会超过480位。

基本的框架搭起来后，再来看错位相加该怎么做呢？到第二轮循环的时候要错开一位和原来c数组存放的结果相加；第三轮循环错开两位再和数组c相加

第一次相乘的结果放到数组c中：`c[j] = a[j] * b[i];`

第二次相乘要错开一位和数组c相加：`c[j+1] = c[j+1] + a[j] * b[i];`

第三次相乘要错开两位和数组c相加：`c[j+2] = c[j+2] + a[j] * b[i];`

每次加完数组c的结果再次存储到数组c中。

这三条语句合起来看：

`c[j+0] = a[j+0] * b[i];`

`c[j+1] = c[j+1] + a[j] * b[i];`

`c[j+2] = c[j+2] + a[j] * b[i];`

发现了，可以用 `j+i` 来替换：`c[j+i] = c[j+i] + a[j] * b[i];`

▼

```

1 // 判断进位
2 void mul(int* arr1, int* arr2, int* ret, char* src1, char* src2)
3 {
4     for (int i = 0; i < strlen(src2); i++)
5     {
6         for (int j = 0; j < strlen(src1); j++)
7         {
8             ret[j + i] = ret[j + i] + (arr1[j] * arr2[i]);
9         }
10    }
11 }

```

现在处理数组c的进位，先来看看当第一次循环结束后，数组c中的数据是什么样的。

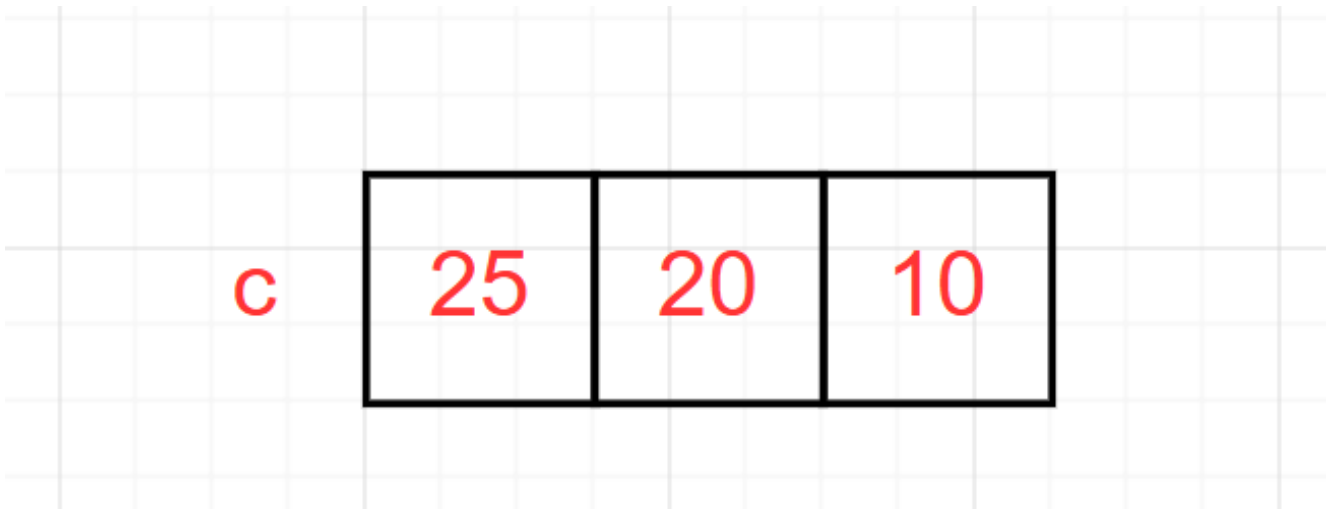


图3

如图3 这是第一次循环存放在数组c的结果（也就是2,4,5分别都乘以5的结果）

`c[0]` 需要向前一个元素进位，留下5；前一个元素 `c[1]` 需要加上进位过来的数字。

遍历数组c，查看每个元素是否需要进位，需要则向前一个元素进位，然后前一个元素加上进位过来的数字。如图4

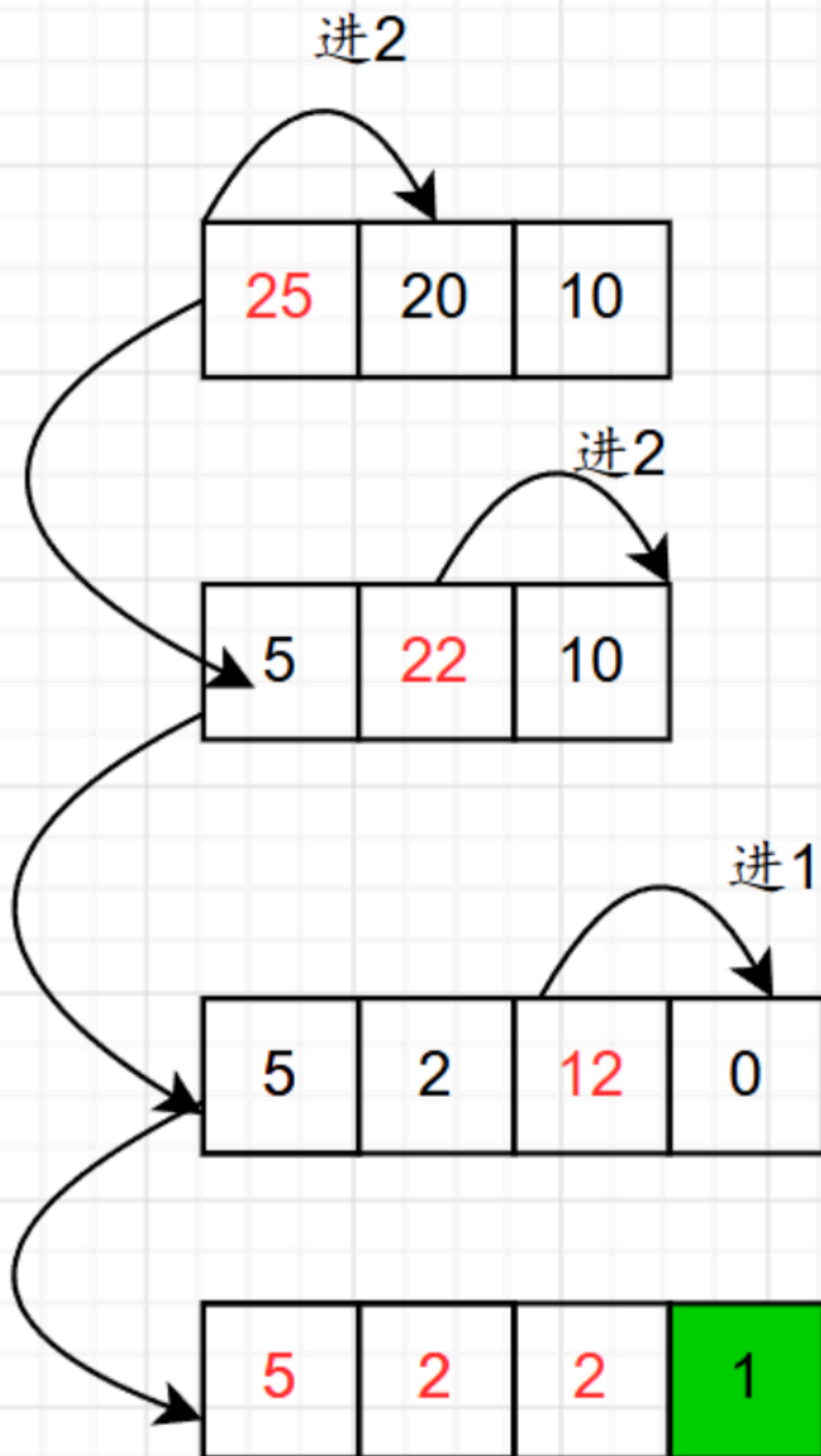


图4

```
1 // 判断进位
2 void mul(int* arr1, int* arr2, int* ret, char* src1, char* src2)
```

```

3 {
4     for (int i = 0; i < strlen(src2); i++)
5     {
6         for (int j = 0; j < strlen(src1); j++)
7         {
8             // 错位相加
9             ret[j + i] = ret[j + i] + (arr1[j] * arr2[i]);
10
11             // 是否进位
12             if (ret[j + i] >= 10)
13             {
14                 ret[j + i + 1] += ret[j + i] / 10;
15                 ret[j + i] = ret[j + i] % 10;
16             }
17         }
18     }
19 }
20 }
21

```

进位的代码可以千万别写错了。我之前写的一个错误版本：

▼ 进位代码错误版本

```

1 // 进位
2 if (ret[j] >= 10)
3 {
4     ret[j+1] = ret[j+1] + ret[j] / 10;
5     ret[j] = ret[j] % 10;
6 }
7

```

假如：s1=125 s2=245

这个判断在第一轮循环的时候是没问题的，但是，在第二轮循环当 `ret[1]=22` 时按理来说要进位，但是判断条件 `ret[j]` 会导致每次都从 `ret[0]` 开始。所以，写成 `ret[j+i]`

其次if代码块的代码也不对，`ret[j+1] = ret[j+1] + ret[j] / 10;` 来套一下，`22 + a[0]/10 = 0`

我的本意是想让22的前一个元素加上22的最高位。`ret[j]` 每轮循环j都是从0开始，这就导致了拿到的不是22这个元素而是 `ret[0]`，这是这条语句的第一个问题，第2个问题是 `ret[j+1]` 这不就是22吗？我自己拿22+2我在干嘛...哈哈 又是造bug的一天

`ret[j] = ret[j] % 10;` 这条语句也就不用多说了也是错的

接着就是倒着输出数组c就好啦，和之前是一样的；先找到倒序输出第一个非0的元素，记录下该位置。两个数相乘其结果的位数不会超过这个两个数的总位数，所以循环条件

`strlen(s1)+strlen(s2)`

```
1 void print(int* ret, size_t len)
2 {
3     int index = 0;
4     for (int i = (int)len - 1; i >= 0; i--)
5     {
6         if (ret[i] != 0)
7         {
8             index = i; // 记录第一个非0的下标位置
9             break;
10        }
11    }
12
13    // 输出结果
14    for (int i = index; i >= 0; i--)
15    {
16        printf("%d", ret[i]);
17    }
18
19 }
```

完整代码：

```
1 #define _CRT_SECURE_NO_WARNINGS 1
2
3
4 #include <stdio.h>
5 #include <string.h>
6
7
8 // 倒序存放
9 void reverse(char* src, int* arr)
10 {
11     for (int i = 0; i < strlen(src); i++)
12     {
13         arr[i] = src[strlen(src) - 1 - i] - '0';
14     }
15 }
16
17 void mul(int* arr1, int* arr2, int* ret, char* src1, char* src2)
18 {
```

```

19 // 控制数组b
20 for (int i = 0; i < strlen(src2); i++)
21 {
22     for (int j = 0; j < strlen(src1); j++) // 控制数组a
23     {
24         // arr1的每一位和arr2第i位相乘
25         ret[j + i] = ret[j + i] + arr1[j] * arr2[i];
26
27         // 进位
28         if (ret[j + i] >= 10)
29         {
30             ret[j + i + 1] += ret[j + i] / 10;
31             ret[j + i] = ret[j + i] % 10;
32         }
33     }
34 }
35 }
36 }
37
38
39
40 void print(int* ret, size_t len)
41 {
42     int index = 0;
43     for (int i = (int)len - 1; i >= 0; i--)
44     {
45         if (ret[i] != 0)
46         {
47             index = i; // 记录第一个非0的下标位置
48             break;
49         }
50     }
51
52     // 输出结果
53     for (int i = index; i >= 0; i--)
54     {
55         printf("%d", ret[i]);
56     }
57
58 }
59
60 int main()
61 {
62     char s1[200] = { 0 };
63     char s2[200] = { 0 };
64
65     int a[210] = { 0 };
66     int b[210] = { 0 };

```

```
67     int c[500] = { 0 };
68     scanf("%s %s", s1, s2);
69
70     // 倒序存放
71     reverse(s1, a);
72     reverse(s2, b);
73
74     mul(a,b,c,s1,s2);
75
76
77     size_t len = strlen(s1) + strlen(s2);
78
79     // 倒序输出
80     print(c, len);
81
82     return 0;
83 }
84
```